

anisotropicLaplacianFoam

Beginner's Guide to Theory, Implementation, Execution, and Tutorial

OpenFOAM v2512 solver documentation

May 25, 2026

Abstract

`anisotropicLaplacianFoam` is an OpenFOAM v2512 solver for transient heat conduction in stationary solid materials with anisotropic thermal conductivity. It solves temperature only. It does not solve fluid velocity, pressure, advection, radiation, or conjugate heat transfer. This document explains the physical model, the OpenFOAM implementation, the required input fields, the execution workflow, and the included tutorial case `tutorials/compositePlate2D`.

Contents

1	What the Solver Does	3
2	Physical Theory	3
2.1	Temperature and Heat Flux	3
2.2	Energy Balance	3
2.3	What Anisotropy Means	4
3	Finite Volume View	4
4	Implementation Layout	5
4.1	Fields Read by <code>createFields.H</code>	5
4.2	Fields Written by <code>write.H</code>	5
5	Build and Run Workflow	6
5.1	Build the Solver	6
5.2	Run the Tutorial	6
6	Tutorial Case Details	7
6.1	Mesh	7
6.2	Boundary Conditions	7
6.3	Material Regions	7
6.4	Internal Heat Generation	8
6.5	Time Controls	8
7	Inspecting Results	8
8	Common Problems	9
9	Minimal Checklist for a New Case	9

1 What the Solver Does

The solver computes how temperature changes in a solid body when heat can conduct through the material and when an optional internal heat source is present. The most important feature is anisotropic conductivity: heat can move more easily in one direction than another.

A common isotropic material uses one conductivity value, for example $k = 1.0 \text{ W}/(\text{m K})$. An anisotropic material uses a tensor:

$$\boldsymbol{\kappa} = \begin{bmatrix} \kappa_{xx} & \kappa_{xy} & \kappa_{xz} \\ \kappa_{xy} & \kappa_{yy} & \kappa_{yz} \\ \kappa_{xz} & \kappa_{yz} & \kappa_{zz} \end{bmatrix}.$$

The tutorial is a thin 2D plate. The left side is held at 300 K, the right side is held at 330 K, the top and bottom are insulated, and a small internal region generates heat.

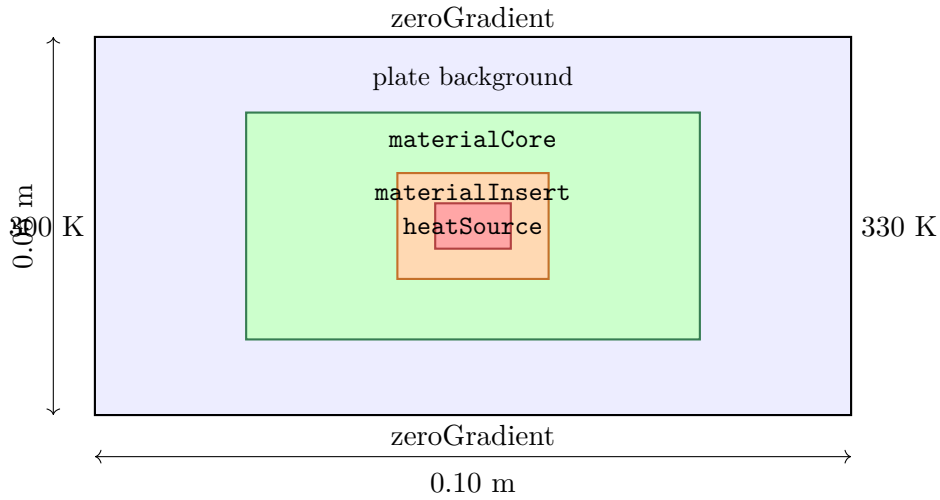


Figure 1: Schematic of the tutorial plate. The actual OpenFOAM mesh has 100 cells in x , 50 cells in y , and one empty cell layer in z .

2 Physical Theory

2.1 Temperature and Heat Flux

Temperature T is the unknown field. Heat flux $\mathbf{q}$ describes the rate and direction of heat flow through a unit area. Fourier's law for anisotropic materials is

$$\mathbf{q} = -\boldsymbol{\kappa} \nabla T.$$

The minus sign is important. The temperature gradient ∇T points toward increasing temperature, but heat naturally flows from hot regions to cold regions. Therefore the heat flux points in the opposite direction after being scaled and rotated by the conductivity tensor.

2.2 Energy Balance

The solver uses the transient heat conduction equation

$$\rho C_p \frac{\partial T}{\partial t} = \nabla \cdot (\boldsymbol{\kappa} \nabla T) + Q.$$

In the code and input files, ρC_p is named **rhoCp**. It is the volumetric heat capacity. It tells the solver how much energy is needed to increase the temperature of one cubic meter of material by one kelvin.

Symbol	OpenFOAM field	SI unit	Meaning
T	T	K	Temperature. This is the solved scalar field.
κ	kappa	W/(m K)	Symmetric thermal conductivity tensor.
ρC_p	rhoCp	J/(m ³ K)	Volumetric heat capacity.
Q	fvOptions	W/m ³	Optional volumetric heat generation.
$\mathbf{q}$	q	W/m ²	Conductive heat flux written by the solver.

Table 1: Main physical quantities used by the solver.

2.3 What Anisotropy Means

In OpenFOAM, a symmetric tensor field is written with six components:

$$(\kappa_{xx} \ \kappa_{xy} \ \kappa_{xz} \ \kappa_{yy} \ \kappa_{yz} \ \kappa_{zz}).$$

For example, the tutorial background material uses

```
internalField    uniform (1.0 0 0 0.3 0 0.3);
```

which means

$$\kappa = \begin{bmatrix} 1.0 & 0 & 0 \\ 0 & 0.3 & 0 \\ 0 & 0 & 0.3 \end{bmatrix}.$$

This material conducts heat more easily in the x direction than in the y or z directions. The tutorial insert material instead uses

$$\kappa = \begin{bmatrix} 0.4 & 0 & 0 \\ 0 & 3.0 & 0 \\ 0 & 0 & 0.4 \end{bmatrix},$$

so heat moves more easily in the y direction inside the insert.

3 Finite Volume View

OpenFOAM uses the finite volume method. The mesh divides the domain into many small control volumes called cells. For each cell, the solver balances the heat stored in the cell, the heat conducted through its faces, and any internal heat generation.

For a beginner, the most useful interpretation is:

- `fvm::ddt(rhoCp, T)` builds the transient heat storage term.
- `fvm::laplacian(kappa, T)` builds the diffusion term caused by anisotropic thermal conductivity.
- `fvOptions(rhoCp, T)` adds optional source terms, such as volumetric heat generation in a selected cell zone.

The algebraic equation solved at every time step is assembled from

```
fvScalarMatrix TEqn
(
    fvm::ddt(rhoCp, T) - fvm::laplacian(kappa, T)
    ==
    fvOptions(rhoCp, T)
);

fvOptions.constrain(TEqn);
TEqn.solve();
fvOptions.correct(T);
```

The code writes derived fields only at output times. The gradient and heat flux are computed as

```
volVectorField gradT(..., fvc::grad(T));
volVectorField q(..., (-kappa & gradT));
```

4 Implementation Layout

The solver intentionally uses a small OpenFOAM application layout. The key files are listed in Table 2.

File	Responsibility
<code>anisotropicLaplacianFoam.C</code>	Main program, time loop, non-orthogonal correction loop, equation assembly, and solve call.
<code>createFields.H</code>	Reads <code>T</code> , <code>kappa</code> , and <code>rhoCp</code> ; creates <code>fvOptions</code> .
<code>write.H</code>	Writes <code>gradT</code> and <code>q</code> at OpenFOAM write times.
<code>Make/files</code>	Defines the solver source and executable name.
<code>Make/options</code>	Defines include paths and OpenFOAM libraries.

Table 2: Source files in the solver.

4.1 Fields Read by `createFields.H`

The solver requires three volume fields in the case time directory, normally 0/. Their OpenFOAM dimensions are shown in Table 3.

Field	OpenFOAM class	Dimensions	Meaning
<code>T</code>	<code>volScalarField</code>	[0 0 0 1 0 0 0]	Temperature in kelvin.
<code>kappa</code>	<code>volSymmTensorField</code>	[1 1 -3 -1 0 0 0]	Thermal conductivity tensor in W/(m K).
<code>rhoCp</code>	<code>volScalarField</code>	[1 -1 -2 -1 0 0 0]	Volumetric heat capacity in J/(m ³ K).

Table 3: Required input fields.

4.2 Fields Written by `write.H`

At every write time, the solver writes the fields in Table 4. These files appear in time directories such as 0.5/, 1/, and 5/, depending on the case settings.

Field	OpenFOAM class	Meaning
T	volScalarField	Solved temperature field.
gradT	volVectorField	Temperature gradient ∇T .
q	volVectorField	Conductive heat flux $\mathbf{q} = -\kappa \nabla T$.

Table 4: Output fields expected from a successful run.

5 Build and Run Workflow

5.1 Build the Solver

Source the OpenFOAM v2512 environment first. The exact command depends on the local installation. After the environment is active, build from the repository root:

```
cd /path/to/anisotropicLaplacianFoam
wmake
```

The executable is installed as $\$(FOAM_USER_APPBIN)/anisotropicLaplacianFoam$, as specified by `Make/files`.

5.2 Run the Tutorial

The tutorial can be run with the provided script:

```
cd tutorials/compositePlate2D
./Allrun
```

The script performs the same workflow shown in Figure 2.

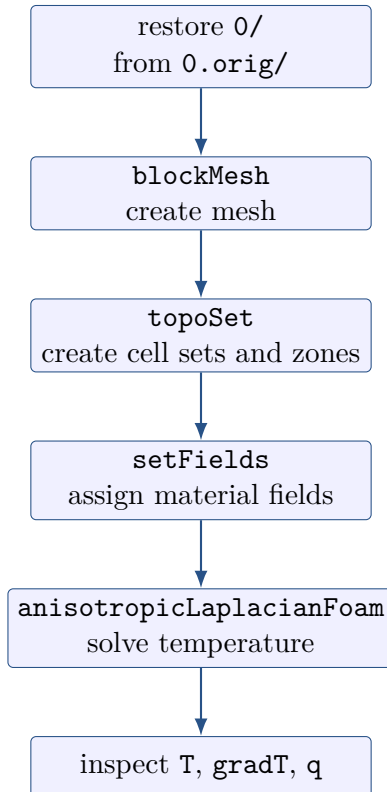


Figure 2: Tutorial execution workflow.

The manual equivalent is:

```
cd tutorials/compositePlate2D
rm -rf 0
cp -r 0.orig 0
blockMesh
topoSet
setFields
anisotropicLaplacianFoam
```

6 Tutorial Case Details

6.1 Mesh

The file `tutorials/compositePlate2D/system/blockMeshDict` creates a rectangular plate:

- length: 0.10 m,
- height: 0.05 m,
- thickness: 0.001 m,
- cells: $100 \times 50 \times 1$.

The front and back patches use the OpenFOAM `empty` boundary type. This is how a geometrically thin 3D mesh is treated as a 2D calculation.

6.2 Boundary Conditions

The initial temperature field is stored in `0.orig/T`. The boundary conditions are:

Patch	Boundary condition	Meaning
<code>left</code>	<code>fixedValue, 300 K</code>	Cold side of the plate.
<code>right</code>	<code>fixedValue, 330 K</code>	Hot side of the plate.
<code>bottom</code>	<code>zeroGradient</code>	Insulated boundary.
<code>top</code>	<code>zeroGradient</code>	Insulated boundary.
<code>frontAndBack</code>	<code>empty</code>	2D front and back patches.

Table 5: Tutorial temperature boundary conditions.

6.3 Material Regions

`topoSet` creates three cell zones:

- `materialCore`: a rectangular core region,
- `materialInsert`: a smaller insert inside the core,
- `heatSource`: a small source region inside the insert.

`setFields` assigns material properties. If a cell is not inside one of the listed regions, it keeps the default background values.

Region	Approximate box in (x, y)	ρC_p	κ components
background	rest of plate	2.0×10^6	(1.0, 0, 0, 0.3, 0, 0.3)
materialCore	(0.02, 0.01) to (0.08, 0.04)	2.4×10^6	(8.0, 0, 0, 0.8, 0, 0.8)
materialInsert	(0.04, 0.018) to (0.06, 0.032)	1.6×10^6	(0.4, 0, 0, 3.0, 0, 0.4)

Table 6: Material properties assigned by `setFieldsDict`. Conductivity components are ordered as (xx, xy, xz, yy, yz, zz) .

6.4 Internal Heat Generation

The file `system/fvOptions` defines a source named `heatGeneration`. Its option type, selected cell zone, and source values are:

```
heatGeneration
{
    type            scalarSemiImplicitSource;
    active          yes;

    selectionMode   cellZone;
    cellZone        heatSource;
    volumeMode      specific;

    injectionRateSuSp
    {
        T          (1.0e6 0);
    }
}
```

The first value is the explicit source coefficient. The second value is the implicit coefficient. In this tutorial the implicit coefficient is zero, so the source behaves as a prescribed heat generation term in the selected zone.

6.5 Time Controls

`system/controlDict` runs from $t = 0$ to $t = 5$ with a time step of $\Delta t = 0.01$. It writes results every 0.5 seconds of simulated time:

```
endTime          5;
deltaT            0.01;
writeControl      runtime;
writeInterval     0.5;
```

7 Inspecting Results

After a successful run, open the case with ParaView or another OpenFOAM-capable post-processor. Useful first checks are:

- Plot `T`. It should remain near the imposed 300 K and 330 K boundary values while being modified by internal heat generation.
- Plot `gradT`. It shows where temperature changes most rapidly.
- Plot `q` as vectors. The vectors show the conductive heat flux direction and magnitude.
- Compare `q` direction in different materials. An anisotropic insert can redirect heat flow even when the temperature contours look smooth.

8 Common Problems

Symptom	Likely cause	Check
Unknown application	The solver has not been built or the OpenFOAM environment is not sourced.	Run <code>wmake</code> from the repository root and check that <code>FOAM_USER_APPBIN</code> is in <code>PATH</code> .
Missing <code>kappa</code> or <code>rhoCp</code>	The case <code>0/</code> directory was not created from <code>0.orig/</code> .	Run <code>rm -rf 0 && cp -r 0.orig 0</code> .
Patch type error for 2D case	One field does not use <code>empty</code> on <code>frontAndBack</code> .	Check <code>T</code> , <code>kappa</code> , and <code>rhoCp</code> .
No heat source effect	<code>topoSet</code> or <code>fvOptions</code> did not create or select <code>heatSource</code> .	Run <code>topoSet</code> ; inspect <code>constant/polyMesh/cellZones</code> .
Unexpected heat flux direction	Sign convention misunderstood.	Remember that <code>q = -kappa & gradT</code> ; heat flows from hot to cold.

Table 7: Beginner troubleshooting checklist.

9 Minimal Checklist for a New Case

To create a new case for this solver:

1. Create a mesh with valid boundary patches.
2. Provide `0/T`, `0/kappa`, and `0/rhoCp`.
3. Use `volSymmTensorField` for `kappa`.
4. Use dimensions `[1 1 -3 -1 0 0 0]` for `kappa`.
5. Use dimensions `[1 -1 -2 -1 0 0 0]` for `rhoCp`.
6. Add `fvOptions` only if an internal heat source is needed.
7. Set `application anisotropicLaplacianFoam`; in `system/controlDict`.
8. Run the solver and inspect `T`, `gradT`, and `q`.

10 Summary

`anisotropicLaplacianFoam` solves transient heat conduction in stationary solids using physical material properties. The required model is compact: `T` is the unknown, `kappa` controls directional heat conduction, and `rhoCp` controls transient heat storage. The tutorial demonstrates the full workflow: create the mesh, create material zones, assign fields, solve, and inspect temperature gradient and heat flux.